

Aula 16

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Formato de Intercâmbio - XML	3
2) Questões Comentadas - Formato de Intercâmbio - XML - Multibancas	16
3) Lista de Questões - Formato de Intercâmbio - XML - Multibancas	26

XML

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTA

O eXtensible Markup Language (XML) pode ser definido como uma metalinguagem de marcação extensível, especificada pela W3C, de propósito geral e que define um conjunto de regras para codificar documentos em um formato que seja legível tanto por humanos quanto por máquinas com o intuito principal de facilitar a representação, armazenamento, transporte e intercâmbio de dados entre sistemas de forma padronizada. *Calma! Nós vamos destrinchar tudo ponto por ponto...*

Em primeiro lugar, trata-se de uma metalinguagem! *O que seria uma metalinguagem?* É basicamente uma linguagem utilizada para descrever outras linguagens. Por exemplo: o idioma português é uma metalinguagem, visto que se trata de uma linguagem capaz de descrever outras linguagens, inclusive ela mesma. É possível definir em português como é a estrutura, gramática, sintaxe, ortografia, etc do próprio português.

O XML é considerado uma metalinguagem porque ele também é autodescritivo, isto é, ele é capaz de descrever a sua própria estrutura. Em segundo lugar, trata-se de uma metalinguagem de marcação. *O que significa isso?* Galera, as marcações (também chamadas de tags) são basicamente anotações (sinais e códigos) dentro de um documento que marcam o início e o fim de um elemento de dados que serão intercambiados – além de idealmente definir seus significados.

Notem por meio do exemplo apresentado a seguir que as marcações são aquelas destacadas em azul e elas apenas marcam o início e fim de elementos de dados – elas não fazem parte do conteúdo em si. Dessa forma, essa linguagem (assim como outras linguagens de marcação) permite representar os dados de tal forma que dados de conteúdo fiquem visualmente separados de dados sobre a estrutura do próprio documento. *Bacana?*

```
<?xml version="1.0" encoding="UTF-8"?>

<carta>
  <de>Banca</de>
  <para>Aluno</para>
  <assunto>Você passou no concurso dos seus sonhos!</assunto>
  <corpo>Isso mesmo que você leu: você está sendo convocado para tomar posse!</corpo>
</carta>
```

Quem aí já ouviu falar em HTML (HyperText Markup Language)? É a linguagem de marcação padrão utilizada na construção de páginas web. *Você acessa páginas web?* Então você acessa uma página escrita (entre outras tecnologias) em HTML. Note que ela também é uma linguagem de marcação, porque ela também tem marcações. Por outro lado, todas as marcações do HTML são pré-definidas, isto é, já existe uma lista de marcações que podem ser definidas.

Já o XML é uma metalinguagem de marcação extensível! *Por que ela é extensível?* Porque as marcações não são pré-definidas – você pode criar suas próprias marcações (tags). Vejam a seguir um exemplo de HTML: observe que se trata também de uma linguagem de marcações, mas as marcações do HTML são pré-definidas. As marcações `<html>`, `<head>`, `<title>`, `<body>`, `<h1>`, `<p>` são pré-definidas na linguagem – já as marcações do XML podem ser criadas pelo usuário...

```
<!DOCTYPE html>
<html>
  <head>
    <title>Título de uma Página Web</title>
  </head>
  <body>
    <h1>Isto é um cabeçalho</h1>
    <p>Isto é um parágrafo</p>
  </body>
</html>
```

Bacana! Nós já entendemos porque XML é uma metalinguagem, porque é uma metalinguagem de marcação e porque é uma linguagem de marcação extensível. *E por que ela é especificada pela W3C?* W3C é a World Wide Web Consortium) – trata-se da principal organização de padronização da World Wide Web. *Por que é bom ser especificada pela W3C?* Porque novas tecnologias e aplicações tentam se adequar às especificações dessa organização. Vamos continuar...

XML é uma linguagem de propósito geral! Nós acabamos de ver que o HTML é uma linguagem que tem basicamente o propósito de definir a estrutura de conteúdos de páginas web, logo se trata de uma linguagem com um propósito bem definido. Já o XML é uma linguagem que tem o propósito de definir a estrutura de intercâmbio de dados de qualquer contexto ou domínio e de forma independente de tecnologia ou plataforma.

O XML define um conjunto de regras para codificar documentos em um formato que seja legível tanto por humanos quanto por máquinas. Essa parte é bacana! Observem o código a seguir...

```
<script type='text/javascript'>
$(document).ready(function()
{
  $('#39;img#closed&#39;).click(function(){$('#39;#bl_banner&#39;).hide(90);});
  document.addEventListener("DOMContentLoaded", function(){
    var popup = document.getElementById("popup");
    var ls = localStorage.getItem("popup");
    var data = new Date();
    var data_atual = data.valueOf();
    var data24 = data.setHours(data.getHours()+24);
    if(ls < data_atual){
      popup.style.display = "block";
      localStorage.setItem("popup", data24);
    }
  });
});
```

Vocês conseguem entendê-lo? Difícil! Por que? Porque esse código foi escrito em uma linguagem chamada JavaScript. Só quem consegue entendê-lo são programadores e, principalmente, aqueles que dominam essa linguagem. Já o XML é uma linguagem que pode ser facilmente lida tanto por

máquinas quanto por humanos, conforme vimos no exemplo da página anterior. *Aquele vocês conseguiram ler e entender, correto? Pois é...*

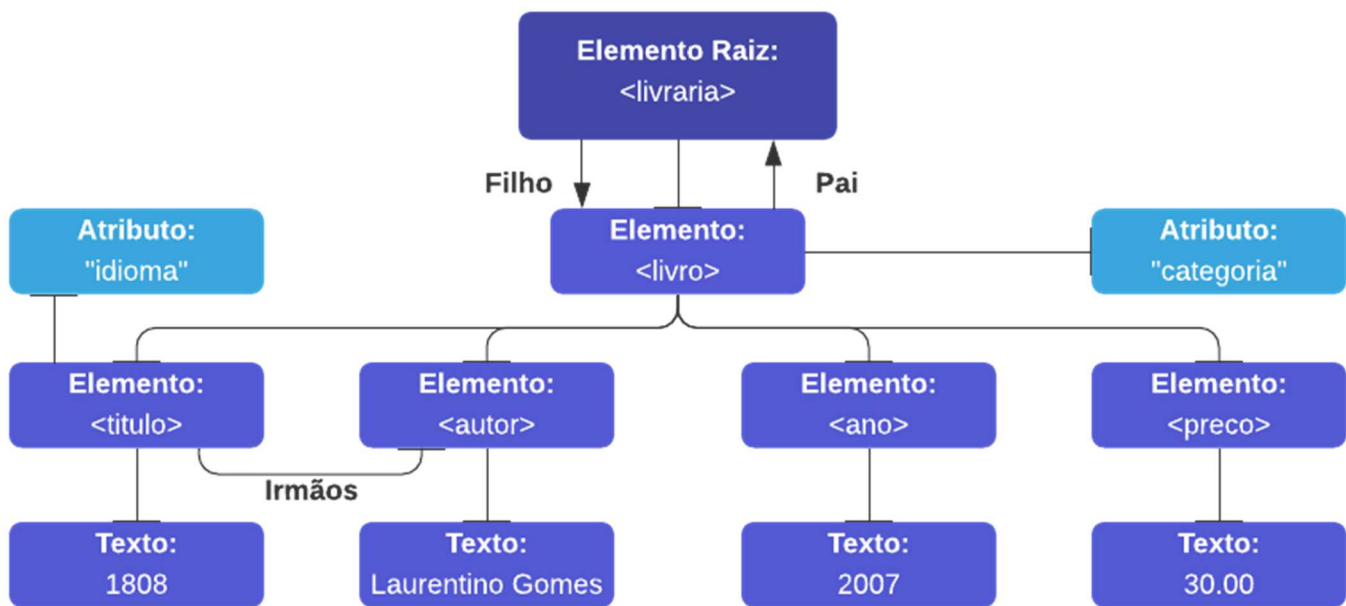
Você bate o olho e consegue identificar que há um emissor de dados, um receptor de dados, um assunto e um corpo contendo o conteúdo em si – é um documento autodescritível. Note que o XML não **executa** nada – ele apenas representa, armazena e facilita o transporte de dados. Quem executa instruções que informam o que o computador deve fazer são as linguagens de programação. XML é uma linguagem de marcação e, não, uma linguagem de programação.

Aliás, essa é a última parte da definição: XML tem o intuito principal de facilitar o intercâmbio de dados entre sistemas de forma padronizada. Trata-se de uma linguagem que busca facilitar o armazenamento, transporte e representação dos dados. Pronto! Terminamos de destrinchar cada ponto da nossa definição. No entanto, ainda não terminamos: vamos falar agora de mais algumas outras características dessa linguagem:

CARACTERÍSTICAS	DESCRIÇÃO
XML SEPARA DADOS DA APRESENTAÇÃO	XML não mantém nenhuma informação sobre como os dados serão exibidos, logo um mesmo documento XML pode ser utilizado em vários cenários de apresentação diferentes.
XML FREQUENTEMENTE COMPLEMENTA O HTML	XML é comumente utilizado para armazenar e transportar dados enquanto HTML é utilizado para formatação e exibição dos mesmos dados – ambos em arquivos separados e tratados independentemente.
XML SUPORTE A UNICODE	XML oferece suporte a Unicode, o que permite a comunicação de quase todas as informações em qualquer linguagem humana escrita.
XML SE ADAPTA A AVANÇOS TECNOLÓGICOS	XML pode se adaptar às novas tecnologias por causa de sua natureza independente de plataforma ou tecnologia. Logo, é uma ótima opção para armazenamento de dados por um longo período.
XML TRATA DADOS EM UMA ESTRUTURA DE ÁRVORE	XML mantém uma estrutura hierárquica de elementos – tanto que o primeiro elemento é sempre o elemento raiz. Isso facilita a representação de dados hierárquicos.
XML É UM FORMATO QUE PODE SER VALIDADO	XML permite fornecer um segundo documento XML – chamado XSD – para descrever exatamente como o arquivo de dados deve ser estruturado, facilitando seu processamento.
XML PERMITE CRIAR OUTRAS LINGUAGENS	XML é uma metalinguagem extensível, logo permite criar outras linguagens. Atualmente, existem linguagens baseadas em XML como WSDL, RSS e XHTML.
XML PERMITE BUSCAS EFICIENTES	Como elementos podem ser unicamente “etiquetados” por meio de tags, isso facilita buscas de dados dentro de documentos.

Estrutura de Árvore

INCIDÊNCIA EM PROVA: BAIXA



Documentos XML são formados como uma árvore de elementos! A imagem acima representa uma estrutura hierárquica de livros que é representada em XML conforme apresenta o código a seguir:

```
<?xml version="1.0" encoding="UTF-8"?>
<livraria>
  <livro categoria="historia">
    <titulo idioma="pt">1808</titulo>
    <autor>Laurentino Gomes</autor>
    <ano>2007</ano>
    <preco>30.00</preco>
  </livro>
  <livro categoria="infantil">
    <titulo idioma="en">Harry Potter</titulo>
    <autor>J K. Rowling</autor>
    <ano>2005</ano>
    <preco>29.99</preco>
  </livro>
  <livro categoria="economia">
    <titulo idioma="pt">A Liberdade</titulo>
    <autor>John Stuart Mill</autor>
    <ano>1859</ano>
    <preco>39.95</preco>
  </livro>
</livraria>
```

Observem que uma árvore sempre começa em um elemento raiz e se ramifica da raiz para os elementos filhos, lembrando que todos os elementos podem ter sub-elementos (elementos filhos). Note que os termos pai, filho e irmãos são utilizados para descrever relacionamentos entre elementos. Pais possuem filhos, filhos possuem pais, irmãos são filhos localizados no mesmo nível. E todos os elementos podem ter conteúdos (Ex: Harry Potter) e atributos (categoria="infantil").

```
<raiz>
  <filho>
    <sub-filho>...</sub-filho>
  </filho>
```

```
</raiz>
```

Diego, por que você pulou a primeira linha do código apresentado na página anterior? Ah, sim! Essa primeira linha (assim como tudo que aparece antes do elemento-raiz) se chama **prolog** e esse componente é responsável por definir a versão do documento, o tipo de codificação de caracteres, instruções de processamento, entre outros. Na verdade, todas essas configurações são opcionais, mas – se existirem – devem ser necessariamente a primeira coisa em um documento.

A declaração do prolog deve ser a primeira coisa em um documento, não pode haver nem um espaço em branco antes. Vamos entendê-la melhor:

```
<?xml version="1.0" encoding="UTF-8"?>
```

Essa linha indica que o documento está utilizando a versão de especificação 1.0 do XML e que os caracteres do arquivo utilizado a codificação UTF-8. *Que codificação é essa, professor?* Galera, documentos podem conter caracteres internacionais como ç, ø, æ, å, è, ß. Para evitar erros, recomenda-se especificar o tipo de codificação de caracteres utilizado. UTF-8 é o tipo de codificação de caracteres padrão do XML (assim como do HTML, SQL, etc). Vamos seguir...

```
<livraria>
  <livro categoria="historia">
    <titulo idioma="pt">1808</titulo>
    <autor>Laurentino Gomes</autor>
    <ano>2007</ano>
    <preco>30.00</preco>
  </livro>
  ...
```

A próxima linha após o prolog contém o elemento raiz do documento: <livraria>; a linha seguinte começa com um elemento <livro>; o elemento livro contém quatro elementos filho: <titulo>, <autor>, <ano>, <preco>; e, por fim, a próxima linha finaliza o elemento <livro> por meio de uma tag de fechamento </livro>. *Conseguiram perceber por que o XML é considerado uma linguagem autodescritiva capaz de ser lida tanto por humanos como por máquinas?* Pois é, vamos continuar...

Sintaxe XML

INCIDÊNCIA EM PROVA: MÉDIA

Elementos

Um elemento é considerado tudo que se encontra entre a tag inicial e uma tag final, incluindo a própria tag do elemento. Ele pode conter outros elementos, textos e atributos – ou também ser vazio. Observem no código seguinte que os elementos <titulo>, <autor>, <ano> e <preco> possuem textos; já os elementos <livro> e <livraria> possuem atributos; e os elementos <livraria> e <livro> possuem outros elementos.

```
<livraria>
```

```
<livro categoria="historia">  
  <titulo idioma="pt">1808</titulo>  
  <autor>Laurentino Gomes</autor>  
  <ano>2007</ano>  
  <preco>30.00</preco>  
</livro>  
</livraria>
```

Um elemento pode ser escrito das duas maneiras apresentadas abaixo – lembrando que todo elemento deve ter uma tag de abertura e de fechamento. *Professor, o prolog apresentado na página anterior não possui nenhuma tag de fechamento!* Sim, bem observado! Isso ocorre por o prolog não é considerado como parte do documento XML em si. Logo, ele não invalida a regra que acabamos de mencionar. *Fechado?*

```
<livro>
```

```
</livro>
```

```
<livro
```

```
/>
```

É possível existir elementos vazios, isto é, aqueles que não possuem nenhum conteúdo (apesar de poder possuir atributos). Eles podem ser representados da seguinte forma:

```
<elemento-vazio></elemento-vazio>
```

É possível também utilizar uma tag auto-fechada, que – na verdade – produz resultados idênticos ao apresentado na linha anterior:

```
<elemento-vazio/>
```

Por fim, existem algumas regras para nomes de elementos: nomes de elementos devem começar com uma letra ou underscore (_); nomes de elementos não podem começar com “xml” ou suas variações de maiúsculas e minúsculas; nomes de elementos podem conter letras, dígitos, hífen, underscore e ponto; nomes de elementos não podem conter espaços. Todo nome pode ser utilizado – não há palavras reservadas (exceto “xml” e suas variações).

Atributos

Atributos são informações adicionais sobre um elemento. Eles vêm dentro da tag de início de um elemento entre aspas (simples ou duplas) e em um formato nome=valor:

```
<pessoa genero="feminino">  
<pessoa genero='feminino'>
```

Se o próprio valor do atributo contiver aspas duplas, você poderá usar aspas simples (apóstrofes) ou símbolos de escape (que veremos em detalhes mais à frente). Vejamos...

```
<lutador nome='Anderson "Spyder" Silva'>  
<lutador nome="Anderson &quot;Spyder&quot; Silva">
```

Agora notem os dois exemplos a seguir: no primeiro, gênero é um atributo; no segundo, é um elemento. Ambos fornecem a mesma informação – não há regras para usar um ou outro!

```
<peessoa genero="feminino">  
  <primeiro-nome>Ana</primeiro-nome>  
  <ultimo-nome>de Amsterdam</ultimo-nome>  
</peessoa>
```

```
<peessoa>  
  <genero>feminino</genero>  
  <primeiro-nome>Ana</primeiro-nome>  
  <ultimo-nome>de Amsterdam</ultimo-nome>  
</peessoa>
```

A única ressalva é que atributos são bem mais limitados que elementos, uma vez que não podem conter múltiplos valores e não podem ser dispostos em uma hierarquia. Além disso, é fundamental destacar um elemento pode conter vários atributos, mas atributos não podem conter múltiplos valores ou estruturas hierárquicas – diferentemente dos elementos. Ademais, dentro de um mesmo elemento, dois atributos jamais podem ter o mesmo nome.

Namespaces

Namespaces são recursos que permitem evitar conflitos de nomes de elementos. Nós vimos que esses nomes são definidos pelo criador do documento, por outro lado frequentemente isso pode resultar em conflitos ao tentar misturar documentos de aplicações diferentes que utilizam o mesmo nome. Vejam no exemplo seguinte dois documentos que utilizam o mesmo nome de elemento, mas um trata do maior clube de futebol do planeta e o outro trata de um bairro do Rio de Janeiro!

```
<flamengo>  
  <esporte>Futebol</esporte>  
  <mascote>Urubu</mascote>  
</flamengo>
```

```
<flamengo>  
  <populacao>50.640</populacao>  
  <distrito>Zona Sul</distrito>  
</flamengo>
```

Se esses documentos interagirem de alguma forma, pode ocorrer um conflito de nomes. Para evitar esse conflito, podemos utilizar prefixos:

```
<raiz>  
  <a:flamengo>  
    <a:esporte>Futebol</a:esporte>  
    <a:mascote>Urubu</a:mascote>  
  </a:flamengo>  
  <b:flamengo>
```

```
<b:populacao>50.640</b:populacao>
<b:distrito>Zona Sul</b:distrito>
</b:flamengo>
</raiz>
```

Ao utilizar um prefixo, um **namespace** deve ser definido. Esse namespace é definido pelo atributo **xmlns** na tag de abertura de um elemento. A declaração do namespace segue a seguinte sintaxe:

```
xmlns:prefix="URI"
```

Vamos ver como é que fica aplicado ao exemplo anterior:

```
<raiz>

<a:flamengo xmlns:a="http://www.flamengo.com.br">
  <a:esporte>Futebol</a:esporte>
  <a:mascote>Urubu</a:mascote>
</a:flamengo>

<b:flamengo xmlns:b="https://prefeitura.rio">
  <b:populacao>50.640</b:populacao>
  <b:distrito>Zona Sul</b:distrito>
</b:flamengo>

</raiz>
```

No exemplo acima, o atributo xmlns no primeiro elemento **<flamengo>** dá um namespace para **a**; já o atributo xmlns no segundo elemento **<flamengo>** dá um namespace para **b**. Lembrando que, quando um namespace é definido para um elemento, todos os elementos filhos com o mesmo prefixo são associados ao mesmo namespace. Além disso, os namespaces também podem ser declarados no elemento raiz do documento.

```
<raiz xmlns:a="http://www.flamengo.com.br"
xmlns:b="https://prefeitura.rio">

<a:flamengo>
  <a:esporte>Futebol</a:esporte>
  <a:mascote>Urubu</a:mascote>
</a:flamengo>

<b:flamengo>
  <b:populacao>50.640</b:populacao>
  <b:distrito>Zona Sul</b:distrito>
</b:flamengo>

</raiz>
```

Professor, fiquei meio perdido! O namespace é um site? Não, o namespace é um nome, mas esse nome precisa ser um identificador único. Logo, é comum a utilização de uma URI (Uniform Resource Identifier) para evitar de ter dois identificadores de namespace com o mesmo nome, mas é completamente possível colocar qualquer identificador desde que ele seja único (Ex: CPF, RG, etc). Entendido, galera? Vamos seguir...

Por fim, é importante mencionar que – quando inserimos um namespace no próprio elemento – podemos evitar de usar os prefixos nos elementos filhos:

```
<flamengo xmlns:a="http://www.flamengo.com.br">
  <esporte>Futebol</esporte>
  <mascote>Urubu</mascote>
</flamengo>

<flamengo xmlns:b="https://prefeitura.rio">
  <populacao>50.640</populacao>
  <distrito>Zona Sul</distrito>
</flamengo>
```

Comentários

A sintaxe para escrever comentários é extremamente simples, só lembrando que não se pode utilizar dois traços no meio do comentário para não confundir o processador do documento:

```
<!-- Isso é um comentário válido -->
<!-- Isso é um comentário -- inválido -->
```

```
<flamengo xmlns:a="http://www.flamengo.com.br">
  <esporte>Futebol</esporte>
  <mascote>Urubu</mascote>
</flamengo>

<!--Flamengo é o maior time de futebol do mundo-->

<flamengo xmlns:b="https://prefeitura.rio">
  <populacao>50.640</populacao>
  <distrito>Zona Sul</distrito>
</flamengo>
```

Espaços em Branco

Em contraste com HTML, XML não trunca ou elimina múltiplos espaços em branco em documentos (Ex: espaços, tabs, quebra de linha). Dessa forma, se você deixar espaços em branco nos seus dados, eles vão permanecer no documento. Vejam os exemplos apresentados no código a seguir e note que temos três registros diferentes. Caso isso fosse um Documento HTML, essa linguagem truncaria (eliminaria) os espaços em branco.

```
<nome>DiegoCarvalho</nome>
<nome>Diego Carvalho</nome>
<nome>Diego          Carvalho</nome>
```

Caracteres Especiais

Nós vimos que os símbolos de menor (<) e maior (>) possuem significados especiais e são utilizados para indicar o início ou fim de uma tag. Se você inserir um caractere como "<" dentro de um

elemento (conforme é apresentado na linha de código apresenta a seguir), isso gerará um erro porque ocasionará uma ambiguidade para o processador¹ do documento que achará que se trata do início de um novo elemento.

```
<concurseiro>salario < 30000</concurseiro>
```

Para evitar esse tipo de problema, existem cinco entidades de escape pré-definidos que ajudam a não confundir o processador do documento:

MENOR QUE	MAIOR QUE	E COMERCIAL	APÓSTROFO	ASPAS
<	>	&	'	"
<	>	&	'	"

É importante destacar que os únicos elementos terminantemente proibidos são < e &. De todo modo, é uma boa prática substituir os outros três pelos seus símbolos correspondentes.

Validação de XML

INCIDÊNCIA EM PROVA: MÉDIA

Dizemos que um documento XML bem formado é aquele que obedece categoricamente às suas regras de sintaxe. Nós já vimos todas as regras, mas vamos lembrá-las:

REGRA 1	Documentos XML devem possuir um único elemento-raiz.
REGRA 2	Todos os elementos devem conter uma <i>tag</i> de fechamento.
REGRA 3	Elementos devem estar corretamente aninhados.
REGRA 4	Atributos devem possuir valor entre aspas simples ou duplas.
REGRA 5	Nomes de tags e atributos são Case-Sensitive.

Legal! Aprendemos o que define um Documento XML bem formado! Agora é o momento de ver o que define um documento XML válido! *São coisas diferentes, professor?* Sim, são conceitos complementares. Todo documento XML válido é bem formado, mas nem todo documento bem formado é válido. O que define a validade de um documento XML não é a sua sintaxe e, sim, se ele é consistente com o seu esquema. *Como é, professor?* Vamos explicar melhor...

Um esquema (ou arquivo de definição) é um modelo que descreve como devem ser os elementos, atributos, dados, entre outros. Um documento XML será considerado válido se ele for bem formado e... obedece às regras de seu esquema. *Vamos abstrair um pouco?* Imaginem o seguinte: eu quero construir um carro! Logo, para que seja um carro bem formado, é preciso que tenha um único volante, quatro rodas, um sistema elétrico e hidráulico, entre outros!

¹ O documento XML é processado por um *parser*, isto é, uma ferramenta que quebra o código em partes menores para verificar sua estrutura gramatical e sintática.



Nós podemos dizer que qualquer veículo que contenha essas características será considerado um carro bem formado. *Fechado?* Mas eu não quero um carro qualquer... eu quero uma Ferrari Califórnia! Logo, para que esse carro seja validado como uma Ferrari Califórnia, é preciso que tenha um motor 4.3L de 460 cv, torque de 77 kgfm, 900 kg, 8 cilindros, transmissão de sete velocidades, suspensão esportiva, design como o da imagem ao lado!

Qualquer objeto que seja considerado um carro bem formado e que contenha essas características que eu defini de forma específica será considerado uma Ferrari Califórnia. Note também que nem todo carro é uma Ferrari Califórnia, mas toda Ferrari Califórnia é um carro. Um outro exemplo que eu gosto de ensinar é a relação entre uma prova discursiva e o padrão de resposta. Muitas vezes, as bancas disponibilizam um padrão de resposta esperada para as provas discursivas.

Nesse contexto, imagine que você fez uma discursiva com o português correto, ortografia impecável, pontuação adequada, sintaxe satisfatória e orações bem construídas – sua prova discursiva estaria bem formada. Agora imaginem que, apesar de tudo isso, seu texto não tem nada a ver com o que o examinador esperava como resposta. Nesse caso, sua prova discursiva está bem formada, mas não é válida. *Entenderam agora?*

Agora vamos voltar para o mundo real: *é obrigatória a utilização de um esquema?* Não! Nós mostramos vários exemplos que não precisavam de esquema nenhum. Em geral, quando se trabalha com documentos pequenos ou se deseja apenas realizar alguns testes, criar um esquema pode ser uma perda de tempo. Já em outros contextos, um esquema permite a verificação da estrutura e regras de preenchimento de um arquivo contendo dados no formato XML.

Dessa forma, ele serve para facilitar a vida do desenvolvedor, ajuda a padronizar documentos para diversos usuários, auxilia o compartilhamento de dados, definem quais tags são válidas e fornece uma boa referência de estrutura para todos que queiram interagir e utilizar os dados. Existem dois tipos principais de arquivos de definição: DTD e XSD! Vamos começar falando do primeiro porque ele é o mais antigo.

O DTD (Document Type Definition) define a estrutura e os elementos/atributos legais permitidos dentro de um documento XML. Trata-se de um conjunto de regras que define quais tipos de dados e entidades farão parte de um documento XML. Estas regras serão utilizadas para que o analisador sintático verifique se o documento é válido ou não. O DTD pode estar definida dentro do próprio arquivo XML ou em um arquivo à parte com extensão .dtd. Vejamos um exemplo anterior:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE carta SYSTEM "Carta.dtd">

<carta>
  <de>Banca</de>
  <para>Aluno</para>
  <assunto>Você passou no concurso dos seus sonhos!</assunto>
  <corpo>Isso mesmo que você leu: você está sendo convocado para tomar posse!</corpo>
</carta>
```

Note que agora a segunda linha contém uma declaração DOCTYPE. *O que isso significa, professor?* Trata-se de uma instrução que faz uma referência a um arquivo DTD externo (no caso, Carta.dtd).

```
<!DOCTYPE carta
[
  <!ELEMENT carta (de,para,assunto,corpo)>
  <!ELEMENT de (#PCDATA)>
  <!ELEMENT para (#PCDATA)>
  <!ELEMENT assunto (#PCDATA)>
  <!ELEMENT corpo (#PCDATA)>
]>
```

Esse arquivo pode ser interpretado da seguinte maneira:

- **!DOCTYPE carta** - define que carta é o elemento raiz do documento;
- **!ELEMENT carta** - define que o elemento carta deve conter esses quatro elementos em ordem;
- **!ELEMENT de** - define o elemento "de" como sendo do tipo #PCDATA;
- **!ELEMENT para** - define o elemento "para" como sendo do tipo #PCDATA;
- **!ELEMENT assunto** - define o elemento "assunto" como sendo do tipo #PCDATA;
- **!ELEMENT corpo** - define o elemento "corpo" como sendo do tipo #PCDATA.

Professor, o que é esse #PCDATA? É um código que indica que determinado elemento contém dados que devem ser processados pelo validador. Existe também o #CDATA, que indica que determinado elemento contém dados que não devem ser processados pelo validador (Ex: caracteres especiais). Por meio do meu arquivo DTD, eu consigo analisar se meu Documento XML é válido. Eu não coloquei no exemplo, mas é possível representar números de ocorrências de um elemento.

Por meio de alguns símbolos: o sinal de mais (+) indica que se trata de um elemento obrigatório que pode aparecer uma ou mais vezes; o sinal de asterisco (*) indica que se trata de um elemento opcional que pode aparecer zero ou mais vezes; o sinal de ponto de interrogação (?) indica que se trata de um elemento opcional que pode aparecer zero ou uma vez; e se não houver símbolo algum, indica que um elemento deve aparecer uma e apenas uma vez. *E o XML Schema?*

Assim como o DTD, o XML Schema Definition (XSD) descreve a estrutura de um documento XML. No entanto, trata-se de uma ferramenta mais poderosa por ser capaz de suportar a criação de namespaces, a definição de novos tipos de dados, a definição de restrições, a conversão de dados, entre outras características que não vamos detalhar. Em contraste com o DTD, ele é escrito em XML – não sendo necessário aprender outra linguagem. Vamos ver um exemplo:

```
<xs:element name="carta">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="de" type="xs:string"/>
      <xs:element name="para" type="xs:string"/>
      <xs:element name="assunto" type="xs:string"/>
      <xs:element name="corpo" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Esse arquivo pode ser interpretado da seguinte maneira:

- `<xs:element name="carta">` define um elemento chamado "carta";
- `<xs:complexType>` "carta" é um elemento do tipo complexo;
- `<xs:sequence>` o tipo complexo é uma sequência de elementos;
- `<xs:element name="de" type="xs:string">` o elemento é uma string;
- `<xs:element name="para" type="xs:string">` o elemento é uma string;
- `<xs:element name="assunto" type="xs:string">` o elemento é uma string;
- `<xs:element name="corpo" type="xs:string">` o elemento é uma string;

QUESTÕES COMENTADAS – DIVERSAS BANCAS

1. (FGV / TCE-AM – 2021) O código XML sintaticamente correto é:

```
<produtos>
  <livro título="Estruturas">
    <ano><2021></ano>
  </livro>
</produtos>
```

a)

```
<produtos>
  <livro título=Estruturas>
    <ano>&<2021&></ano>
  </livro>
</produtos>
```

b)

```
<produtos>
  <livro título="Estruturas">
    <ano>2021<ano>
  <livro>
</produtos>
```

c)

```
<produtos>
  <livro título=Estruturas>
    <ano>"<2021>"</ano>
  </livro>
</produtos>
```

d)

```
<produtos>
  <livro título="Estruturas">
    <ano>&lt;2021&gt;</ano>
  </livro>
</produtos>
```

e) </produtos>

Comentários:

(a) Errado, os caracteres "<" e ">" precisam ser escapados; (b) Errado, faltam as aspas em "Estruturas"; (c) Errado, faltou fechar a tag </ano> e a tag </livro>; (d) Errado, faltam as aspas em "Estruturas"; (e) Correto. Alguns caracteres especiais devem ser especificados com o uso de entidades pré-definidas (no caso <, & e >, respectivamente).

Gabarito: Letra E

2. (VUNESP / TJM-SP – 2021) No XML, os nomes de elementos:

- a) não diferenciam maiúsculas de minúsculas.
- b) devem ser iniciados com um caractere letra ou sublinhado.

- c) podem conter letras, números, hifens, sublinhados, pontos ou espaços.
- d) não podem conter caracteres acentuados.
- e) não podem fazer uso de nomes existentes no HTML.

Comentários:

(a) Errado, XML é case-sensitive; (b) Correto; (c) Errado, podem conter letras, números, hifens, sublinhados e pontos – espaço, não; (d) Errado, podem – sim – conter caracteres acentuados; (e) Errado, podem sem nenhum problema.

Gabarito: Letra B

3. (FGV / IMBEL – 2021) O uso do XML é bastante difundido no Brasil para troca de dados em aplicações como notas fiscais, procedimentos médicos, e várias outras. Assinale a utilidade de um Schema XML em aplicações dessa natureza.

- a) Direciona os aplicativos que leem arquivos no formato XML.
- b) Permite a conversão automática de arquivos XML para o formato CSV.
- c) Estabelece precisamente a versão do XML em uso num arquivo no formato XML.
- d) Permite que um Web Service interprete corretamente um arquivo de dados no formato XML.
- e) Permite a verificação da estrutura e regras de preenchimento de um arquivo contendo dados no formato XML.

Comentários:

(a) Errado, isso é uma função do sistema operacional; (b) Errado, essa definitivamente não é uma aplicação do XML Schema; (c) Errado, isso é função do prolog e, não, do XML Schema; (d) Errado, essa não é uma de suas funções; (e) Correto, o XML Schema realmente permite a verificação da estrutura e regras de preenchimento de um arquivo contendo dados no formato XML.

Gabarito: Letra E

4. (VUNESP / SEMAE DE PIRACICABA – 2021) A opção que representa a forma correta de inserção de um comentário dentro de um arquivo XML é:

- a) // meu comentário
- b) <!-- meu comentário -->
- c) /* meu comentário */
- d) # meu comentário
- e) ** meu comentário

Comentários:

Comentários utilizam a sintaxe: <!-- meu comentário -->

Gabarito: Letra B

5. (IDIB / CRF-MS – 2021) Em relação ao XML, analise as afirmativas a seguir:

- (I) O XML é uma linguagem de marcação como o HTML.
- (II) O XML é uma linguagem de programação para ser compilada.
- (III) O XML é utilizado para armazenar e transportar dados.

É correto o que se afirma:

- a) apenas em I e II.
- b) apenas em II e III.
- c) apenas em I.
- d) apenas em I e III.

Comentários:

(I) Correto; (II) Errado, não se trata de uma linguagem de programação e, sim, uma linguagem de marcação; (III) Correto.

Gabarito: Letra D

6. (VUNESP / Prefeitura de Presidente Prudente-SP – 2021) Segundo a especificação do XML, se um documento XML é considerado válido, então, é correto afirmar que:

- a) ele também é bem-formatado.
- b) ele possui uma Definição de Tipo de Documento (DTD), mas a sintaxe não necessariamente está correta.
- c) a sintaxe dele está correta, mas ele não possui uma Definição de Tipo de Documento (DTD).
- d) todos os elementos que compõem o documento possuem, no máximo, um único elemento filho.
- e) todos os elementos possuem a propriedade "id" e estão corretamente identificados.

Comentários:

(a) Correto, um documento válido necessariamente é bem formado; (b) Errado, ele não precisa ter necessariamente um DTD; (c) Errado, ele pode ou não possuir um DTD; (d) Errado, isso não faz qualquer sentido lógico; (e) Errado, não é obrigatório ter uma propriedade "id".

Gabarito: Letra A

7. (COMPERVE / TJ-RN – 2020) Alguns caracteres causam problemas quando são colocados dentro de conteúdo ou como valores de atributos no XML. Por isso, certos caracteres são proibidos na linguagem, tais como:

- a) = e "
- b) < e !
- c) ! e =
- d) " e <

Comentários:

Os caracteres terminantemente proibidos são < e &; outros que a documentação recomenda veemente que não se utilizem são >, ' e ".

Gabarito: Letra D

8. (FGV / MPE-RJ – 2019) A troca de dados entre sistemas computacionais é normalmente realizada por meio de arquivos que seguem padrões de formato e organização. Desse modo, diferentes agentes com diferentes equipamentos podem enviar e receber dados estruturados muito facilmente. Nesse contexto, analise um trecho do conteúdo de um dado arquivo a seguir.

```
<nota>  
    <para>Rita</para>  
    <de>Bernardo</de>  
    <título>Lembrete</título>  
    <texto>O pacote &lt;chegou> ...</texto>  
</nota>
```

Com base nesse trecho, é correto deduzir que a organização desse arquivo segue o padrão conhecido como:

- a) CSS.
- b) CSV.
- c) ODF.
- d) PDF.
- e) XML.

Comentários:

(a) Errado, CSS é uma linguagem de estilo de páginas web; (b) Errado, CSV é um formato de dados tabulares separados por um delimitador; (c) Errado, ODF é um formato para criação de arquivos do LibreOffice; (d) Errado, PDF é um formato de documentos portáteis; (e) Correto.

Gabarito: Letra E

9. (FGV / DPE-RJ – 2019) Considere os trechos XML exibidos a seguir.

- I.
<p>Um primeiro exemplo</p>

- II.
<message>Texto breve</message>
- III.
<i>Texto com destaque.</i>
- IV.
<p>Note que, para x>1, a resposta é sim.</p>

O número de trechos válidos é:

- a) 0.
- b) 1.
- c) 2.
- d) 3.
- e) 4.

Comentários:

(I) Errado, não pode haver elementos após o fechamento do elemento raiz; (II) Correto; (III) Errado, as tags não estão aninhadas corretamente; (IV) Errado, recomenda-se que o caractere ">" seja escapado por meio de um >.

Gabarito: Letra B

10. (AOCF / UFPB – 2019) O XML não é uma linguagem de programação, mas, sim, de marcação. Sobre XML, é correto afirmar que:

- a) o XML é utilizado para aumentar a velocidade na troca de informação.
- b) o XML pode ser definido como uma linguagem de metamarcação.
- c) o XML nada mais é do que um arquivo que contém a codificação de exibição de uma página web.
- d) o XML é uma linguagem de orientação a objetos.
- e) o XML pode ser conceituado como uma linguagem de metaprogramação.

Comentários:

(a) Errado, não há nada que justifique uma alternativa como essa sem um parâmetro de comparação – por exemplo: ele é mais lento que o JSON; (b) Correto, pode ser considerado uma linguagem de metamarcação, dado que ela pode utilizar marcações para definir as próprias marcações; (c) Errado, trata-se de uma linguagem para o armazenamento, compartilhamento, e intercâmbio de dados; (d) Errado, não se trata de uma linguagem orientada a objetos – sequer existe o conceito de objeto em XML; (e) Errado, metaprogramação é a programação de programas que se autoprogramam ou programam outros programas, não há nenhuma relação com XML.

Gabarito: Letra B

11. (CVV / UFC – 2019) Sobre as características da linguagem XML (eXtensible Markup Language), é correto afirmar:

- a) o usuário da linguagem XML pode definir novas tags para melhor estruturar a informação contida no arquivo.
- b) a XML é uma linguagem de programação que necessita de um compilador específico para gerar o arquivo binário a ser executado em algum sistema.
- c) a desvantagem da XML é a dependência da plataforma sobre a qual está executando, sendo necessárias adequações para cada tipo de sistema.
- d) a característica de extensibilidade da linguagem está relacionada ao fato de ser possível criar funções a partir de um conjunto fixo de tags fornecido pela linguagem.
- e) a linguagem XML fornece o recurso de tipagem dos dados, de forma que, por exemplo, é possível definir números inteiros e realizar operações sobre eles no programa XML.

Comentários:

(a) Correto, trata-se de uma linguagem extensível; (b) Errado, trata-se de uma linguagem de marcação e, não, de programação; (c) Errado, ele é independente de plataforma; (d) Errado, ela é considerada extensível porque permite estender a quantidade de tags, dado que cada usuário pode criar suas tags sem limitações; (e) Errado, ela não é uma linguagem de programação, logo não há que se falar em operações.

Gabarito: Letra A

12. (FGV / MPE-AL – 2018) No XML, a sequência de símbolos

<

Representa:

- a) <
- b) >
- c) "
- d) &
- e) '

Comentários:

MENOR QUE	MAIOR QUE	E COMERCIAL	APÓSTROFO	ASPAS
<	>	&	'	"
<	>	&	'	"

Gabarito: Letra A

13. (CCV / UFC – 2018) Qual dos seguintes fragmentos representa um fragmento XML bem formado?

- a) <myElement myAttribute="someValue"/>
- b) <myElement myAttribute="someValue'/'>
- c) <myElement myAttribute='someValue'>
- d) <myElement myAttribute=someValue/>
- e) <myElement myAttribute=someValue>

Comentários:

(a) Correto; (b) Errado, se abriu com aspas duplas deve fechar com aspas duplas; (c) Errado, a tag de fechamento está incorreta – deveria ser />; (d) Errado, utilizam-se aspas simples ou duplas; (e) Errado, utilizam-se aspas simples ou duplas – e a tag de fechamento também está incorreta.

Gabarito: Letra A

14. (FGV / Câmara de Salvador-BA – 2018) Analise o conteúdo XML de um arquivo de seis linhas, exibido a seguir.

```
<?xml version="1.0" encoding="UTF-8"?>
<database catalogo="BD" user="U1">
  <SQL>
    select * FROM T where a < 10
  </SQL>
</database>
```

A validação desse arquivo apontaria um erro na linha de número:

- a) 1.
- b) 2.

- c) 3.
- d) 4.
- e) 6.

Comentários:

Ocorrerá um erro na Linha 4, dado que é recomendado escapar o caractere < por meio de um <.

Gabarito: Letra D

15. (CESGRANRIO / TRANSPETRO – 2018) Considerando a linguagem XML, qual é o exemplo correto de uso de um atributo chamado "src" que recebe o valor "computador.gif" em um elemento de nome "img"?

- a)
- b)
- c) "src=computador.gif"
- d) <src> computador.gif </src>
- e) src="computador.gif "

Comentários:

(a) Correto; (b) Errado, a tag de fechamento deve ser /> ou ; (c) Errado, a tag de abertura deve ser <img e, não, ; (d) Errado, src é um atributo e, não, um elemento; (e) Errado, a tag de abertura deve ser <img e, não, .

Gabarito: Letra A

16. (QUADRIX / CRQ4-SP – 2018) Uma vantagem do DTD é que ele é escrito em linguagem XML, enquanto o XML-Schema possui outra sintaxe de programação.

Comentários:

DTD não é escrito em XML! Já o XML Schema é escrito em XML, mas utiliza uma sintaxe de marcação e, não, de programação.

Gabarito: Errado

17. (CESPE / Polícia Federal - 2018) Em arquivos no formato XML, as tags não são consideradas metadados.

Comentários:

As tags são consideradas metadados porque fornecem informações sobre os dados que elas envolvem e dados sobre dados são metadados.

Gabarito: Errado

18.(QUADRIX / CRO4-SP – 2018) Para a descrição de um XML, tanto o XML-Schema quanto a DTD (Document Type Definition) podem definir os elementos e atributos que podem aparecer em um documento, os tipos de dados para elementos e atributos e os valores-padrão e fixos para elementos e atributos.

Comentários:

Perfeito! O DTD é mais antigo e o XML Schema é mais novo e poderoso, mas ambos podem ser utilizados para descrever um XML.

Gabarito: Correto

19.(CESGRANRIO / PETROBRAS – 2018) Qual linguagem de marcação, fundamental para o estabelecimento de serviços Web, que compõe uma Arquitetura Orientada a Serviços, é usada para que dados sejam apresentados, comunicados e armazenados?

- a) HTML;
- b) XML;
- c) JAVA;
- d) JAVASCRIPT;
- e) C#.

Comentários:

A linguagem utilizada para apresentar, comunicar e armazenar dados é o XML.

Gabarito: Letra B

20.(FGV / Prefeitura de Niterói-RJ – 2018) Considere a declaração do tipo de documento (DTD) a seguir.

```
<!ELEMENT blog (nome, autor+, artigo*, permalink?) >  
<!ELEMENT nome (#PCDATA)>  
<!ELEMENT autor (#PCDATA)>  
<!ELEMENT artigo (#PCDATA)>  
<!ELEMENT permalink (#PCDATA)>
```

Em um documento XML, que obedece a esse conjunto de regras,

- a) deve haver precisamente um elemento do tipo artigo.
- b) podem ocorrer vários elementos do tipo permalink.
- c) pode conter um elemento do tipo nome e outro elemento do tipo autor, em qualquer ordem.
- d) deve existir um elemento do tipo autor, sendo permitidas múltiplas ocorrências deste tipo de elemento.
- e) não é necessário haver um elemento do tipo nome.

Comentários:

(a) Errado, o asterisco indica que o elemento pode aparecer zero ou mais vezes; (b) Errado, a interrogação indica que o elemento deve aparecer zero ou uma vez; (c) Errado, a ordem dos elementos entre parênteses deve ser respeitada; (d) Correto, o sinal de mais indica que o elemento pode aparecer uma ou mais vezes; (e) Errado, quando não há nenhum símbolo, indica que deve haver necessariamente um único elemento.

Gabarito: Letra D

▪

LISTA DE QUESTÕES – DIVERSAS BANCAS

1. (FGV / TCE-AM – 2021) O código XML sintaticamente correto é:

```
<produtos>
  <livro título="Estruturas">
    <ano><2021></ano>
  </livro>
</produtos>
```

a)

```
<produtos>
  <livro título=Estruturas>
    <ano>&<2021&></ano>
  </livro>
</produtos>
```

b)

```
<produtos>
  <livro título="Estruturas">
    <ano>2021<ano>
  <livro>
</produtos>
```

c)

```
<produtos>
  <livro título=Estruturas>
    <ano>"<2021>"</ano>
  </livro>
</produtos>
```

d)

```
<produtos>
  <livro título="Estruturas">
    <ano>&lt;2021&gt;</ano>
  </livro>
</produtos>
```

e) </produtos>

2. (VUNESP / TJM-SP – 2021) No XML, os nomes de elementos:

- a) não diferenciam maiúsculas de minúsculas.
- b) devem ser iniciados com um caractere letra ou sublinhado.
- c) podem conter letras, números, hifens, sublinhados, pontos ou espaços.
- d) não podem conter caracteres acentuados.
- e) não podem fazer uso de nomes existentes no HTML.

3. (FGV / IMBEL – 2021) O uso do XML é bastante difundido no Brasil para troca de dados em aplicações como notas fiscais, procedimentos médicos, e várias outras. Assinale a utilidade de um Schema XML em aplicações dessa natureza.

- a) Direciona os aplicativos que leem arquivos no formato XML.

- b) Permite a conversão automática de arquivos XML para o formato CSV.
- c) Estabelece precisamente a versão do XML em uso num arquivo no formato XML.
- d) Permite que um Web Service interprete corretamente um arquivo de dados no formato XML.
- e) Permite a verificação da estrutura e regras de preenchimento de um arquivo contendo dados no formato XML.

4. (VUNESP / SEMAE DE PIRACICABA – 2021) A opção que representa a forma correta de inserção de um comentário dentro de um arquivo XML é:

- a) // meu comentário
- b) <!-- meu comentário -->
- c) /* meu comentário */
- d) # meu comentário
- e) ** meu comentário

5. (IDIB / CRF-MS – 2021) Em relação ao XML, analise as afirmativas a seguir:

- (I) O XML é uma linguagem de marcação como o HTML.
- (II) O XML é uma linguagem de programação para ser compilada.
- (III) O XML é utilizado para armazenar e transportar dados.

É correto o que se afirma:

- a) apenas em I e II.
- b) apenas em II e III.
- c) apenas em I.
- d) apenas em I e III.

6. (VUNESP / Prefeitura de Presidente Prudente-SP – 2021) Segundo a especificação do XML, se um documento XML é considerado válido, então, é correto afirmar que:

- a) ele também é bem-formatado.
- b) ele possui uma Definição de Tipo de Documento (DTD), mas a sintaxe não necessariamente está correta.
- c) a sintaxe dele está correta, mas ele não possui uma Definição de Tipo de Documento (DTD).
- d) todos os elementos que compõem o documento possuem, no máximo, um único elemento filho.
- e) todos os elementos possuem a propriedade "id" e estão corretamente identificados.

7. (COMPERVE / TJ-RN – 2020) Alguns caracteres causam problemas quando são colocados dentro de conteúdo ou como valores de atributos no XML. Por isso, certos caracteres são proibidos na linguagem, tais como:

- a) = e "

- b) < e !
- c) ! e =
- d) " e <

8. (FGV / MPE-RJ – 2019) A troca de dados entre sistemas computacionais é normalmente realizada por meio de arquivos que seguem padrões de formato e organização. Desse modo, diferentes agentes com diferentes equipamentos podem enviar e receber dados estruturados muito facilmente. Nesse contexto, analise um trecho do conteúdo de um dado arquivo a seguir.

```
<nota>
    <para>Rita</para>
    <de>Bernardo</de>
    <título>Lembrete</título>
    <texto>O pacote &lt;chegou> ...</texto>
</nota>
```

Com base nesse trecho, é correto deduzir que a organização desse arquivo segue o padrão conhecido como:

- a) CSS.
- b) CSV.
- c) ODF.
- d) PDF.
- e) XML.

9. (FGV / DPE-RJ – 2019) Considere os trechos XML exibidos a seguir.

- I.
<p>Um primeiro exemplo</p>

- II.
<message>Texto breve</message>
- III.
<i>Texto com destaque.</i>
- IV.
<p>Note que, para x>1, a resposta é sim.</p>

O número de trechos válidos é:

- a) 0.
- b) 1.
- c) 2.
- d) 3.

e) 4.

10. (AOCP / UFPB – 2019) O XML não é uma linguagem de programação, mas, sim, de marcação. Sobre XML, é correto afirmar que:

- a) o XML é utilizado para aumentar a velocidade na troca de informação.
- b) o XML pode ser definido como uma linguagem de metamarcação.
- c) o XML nada mais é do que um arquivo que contém a codificação de exibição de uma página web.
- d) o XML é uma linguagem de orientação a objetos.
- e) o XML pode ser conceituado como uma linguagem de metaprogramação.

11. (CVV / UFC – 2019) Sobre as características da linguagem XML (eXtensible Markup Language), é correto afirmar:

- a) o usuário da linguagem XML pode definir novas tags para melhor estruturar a informação contida no arquivo.
- b) a XML é uma linguagem de programação que necessita de um compilador específico para gerar o arquivo binário a ser executado em algum sistema.
- c) a desvantagem da XML é a dependência da plataforma sobre a qual está executando, sendo necessárias adequações para cada tipo de sistema.
- d) a característica de extensibilidade da linguagem está relacionada ao fato de ser possível criar funções a partir de um conjunto fixo de tags fornecido pela linguagem.
- e) a linguagem XML fornece o recurso de tipagem dos dados, de forma que, por exemplo, é possível definir números inteiros e realizar operações sobre eles no programa XML.

12. (FGV / MPE-AL – 2018) No XML, a sequência de símbolos

<

Representa:

- a) <
- b) >
- c) "
- d) &

13. (CCV / UFC – 2018) Qual dos seguintes fragmentos representa um fragmento XML bem formado?

- a) <myElement myAttribute="someValue"/>
- b) <myElement myAttribute="someValue' />
- c) <myElement myAttribute='someValue'>
- d) <myElement myAttribute=someValue/>
- e) <myElement myAttribute=someValue>

14.(FGV / Câmara de Salvador-BA – 2018) Analise o conteúdo XML de um arquivo de seis linhas, exibido a seguir.

```
<?xml version="1.0" encoding="UTF-8"?>
<database catalogo="BD" user="U1">
  <SQL>
    select * FROM T where a < 10
  </SQL>
</database>
```

A validação desse arquivo apontaria um erro na linha de número:

- a) 1.
- b) 2.
- c) 3.
- d) 4.
- e) 6.

15.(CESGRANRIO / TRANSPETRO – 2018) Considerando a linguagem XML, qual é o exemplo correto de uso de um atributo chamado "src" que recebe o valor "computador.gif" em um elemento de nome "img"?

- a)
- b)
- c) "src=computador.gif"
- d) <src> computador.gif </src>
- e) src="computador.gif "

16.(QUADRIX / CRQ4-SP – 2018) Uma vantagem do DTD é que ele é escrito em linguagem XML, enquanto o XML-Schema possui outra sintaxe de programação.

17.(CESPE / Polícia Federal - 2018) Em arquivos no formato XML, as tags não são consideradas metadados.

18.(QUADRIX / CRQ4-SP – 2018) Para a descrição de um XML, tanto o XML-Schema quanto a DTD (Document Type Definition) podem definir os elementos e atributos que podem aparecer em um documento, os tipos de dados para elementos e atributos e os valores-padrão e fixos para elementos e atributos.

19.(CESGRANRIO / PETROBRAS – 2018) Qual linguagem de marcação, fundamental para o estabelecimento de serviços Web, que compõe uma Arquitetura Orientada a Serviços, é usada para que dados sejam apresentados, comunicados e armazenados?

- a) HTML;
- b) XML;
- c) JAVA;
- d) JAVASCRIPT;
- e) C#.

20.(FGV / Prefeitura de Niterói-RJ – 2018) Considere a declaração do tipo de documento (DTD) a seguir.

```
<!ELEMENT blog (nome, autor+, artigo*, permalink?) >  
<!ELEMENT nome (#PCDATA)>  
<!ELEMENT autor (#PCDATA)>  
<!ELEMENT artigo (#PCDATA)>  
<!ELEMENT permalink (#PCDATA)>
```

Em um documento XML, que obedece a esse conjunto de regras,

- a) deve haver precisamente um elemento do tipo artigo.
- b) podem ocorrer vários elementos do tipo permalink.
- c) pode conter um elemento do tipo nome e outro elemento do tipo autor, em qualquer ordem.
- d) deve existir um elemento do tipo autor, sendo permitidas múltiplas ocorrências deste tipo de elemento.
- e) não é necessário haver um elemento do tipo nome.

GABARITO – DIVERSAS BANCAS

- | | |
|-------------|-------------|
| 1. LETRA E | 11. LETRA A |
| 2. LETRA B | 12. LETRA A |
| 3. LETRA E | 13. LETRA A |
| 4. LETRA B | 14. LETRA D |
| 5. LETRA D | 15. LETRA A |
| 6. LETRA A | 16. ERRADO |
| 7. LETRA D | 17. ERRADO |
| 8. LETRA E | 18. CORRETO |
| 9. LETRA B | 19. LETRA B |
| 10. LETRA B | 20. LETRA D |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.